REINFORCEMENT LEARNING PROJECT · CRAFTER (HAFNER, 2022)

Chasing the diamond in Crafter

PPO with Achievement Distillation, plus a **rarity bonus** that rewards the achievements the agent almost never gets. 1M environment steps, 3 seeds per configuration.

23.65 ± 1.05
Crafter score, ours (AD + rarity β=2)

15.75 ± 2.99
PPO baseline (paper: 15.60 ± 1.66)

1.55
uniformly random agent

Barbu Alexandru · November 2026 · code: `python train.py` trains the best configuration

The task: Crafter, a 2D Minecraft with a 1M-step budget

Every episode is a new procedurally generated world. The agent sees 64×64 pixels and has 17 actions.

- **22 achievements** form a tech tree: wood → table → wood pickaxe → stone → stone pickaxe → coal + furnace → iron → iron pickaxe → **diamond**. Survival needs food, water, sleep and fighting zombies at night.
- **Reward:** +1 the first time each achievement is unlocked in an episode, ±0.1 per health point gained or lost.
- **Score:** success rate s_i = % of all training episodes that unlocked achievement i, then a geometric mean:

$$S = \exp\left(\frac{1}{22}\sum_{i=1}^{22}\ln(1+s_i)\right) - 1$$

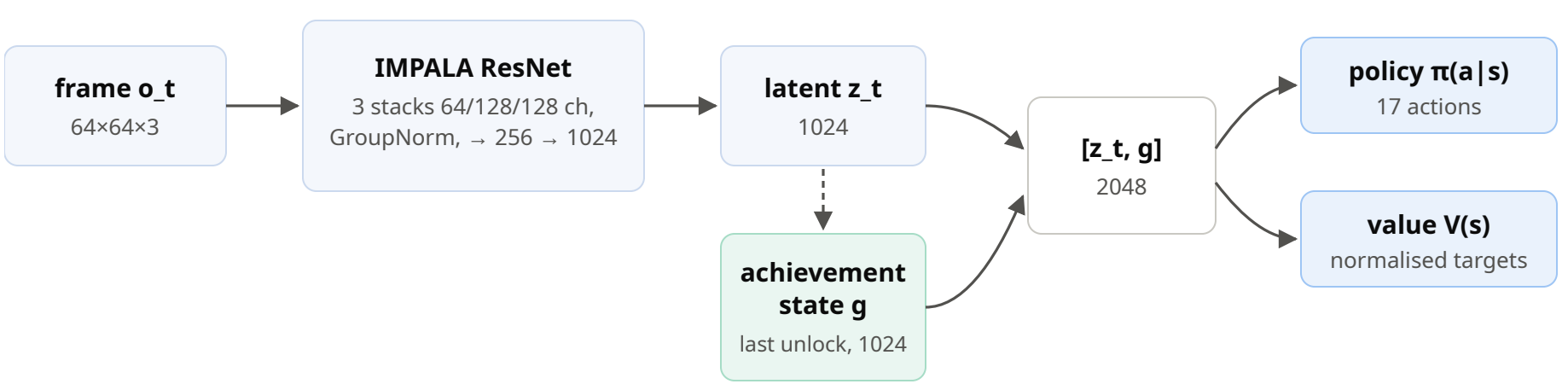
- The geometric mean is **dominated by the rare achievements**: taking one from 0% to 1% helps the score as much as taking another from 10% to 21%. This is the idea behind our change.
- Protocol: 1M steps of training, score over *all* training episodes, mean of 3 seeds.



Our agent (seed 0, final weights) mines iron and makes an iron pickaxe at step 178, at night.

Method: PPO with Achievement Distillation (AD)

AD (Moon et al., NeurIPS 2023) is the strongest published model-free method on Crafter. We re-implemented it and built on it.



g = normalise(enc(o after unlock) − enc(o before)),
kept until the next unlock, zero at episode start

reward to learner

game reward r_t
+ β(1 − p_i) on a
first unlock of i (ours)

AD auxiliary phase

every 8 PPO updates:
contrastive prediction +
OT matching of achievements

Loop. 8 envs × 512 steps = 4,096 steps per update, 245 updates = 1M steps. GAE returns (γ = 0.95, λ = 0.65), then PPO: 3 epochs × 8 minibatches of 512, Adam 3e-4, grad-norm 0.5.

Auxiliary phase (every 8 updates, 6 epochs on the last 8 rollouts split into whole episodes): shapes the encoder so that achievements have consistent embeddings, while KL and value terms keep π and V in place.

Objective functions

PPO every update; the two AD losses in the auxiliary phase. Each AD loss has its own Adam optimiser.

PPO (clipped surrogate, GAE)

$$L_{\text{PPO}} = -\,\mathbb{E}_t\Big[\min(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t)\Big] + c_v\,\mathbb{E}_t\Big[(V_\theta(s_t) - \hat{R}_t)^2\Big] - c_e\,\mathbb{E}_t\Big[\mathcal{H}(\pi_\theta(\cdot|s_t))\Big]$$

$\rho_t = \pi_\theta(a_t|s_t)/\pi_{\text{old}}(a_t|s_t)$, $\epsilon = 0.2,\ c_v = 0.5,\ c_e = 0.01$. Value targets are normalised by a running mean / variance.

$$\hat{A}_t = \textstyle\sum_{l\geq 0}(\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad \hat{R}_t = \hat{A}_t + V(s_t)$$

Reward the learner sees (ours)

$$r'_t = r_t + \beta\,(1 - p_i)$$

added on the step where achievement i is first unlocked in the episode; p_i = share of the last 200 episodes that unlocked i; $\beta = 2$.

AD: intra-episode prediction

$$L_{\text{pred}} = -\log \frac{e^{\langle \hat{g}(z_t,a_t),\, g^+\rangle/\tau}}{e^{\langle \hat{g}(z_t,a_t),\, g^+\rangle/\tau} + e^{\langle \hat{g}(z_{t'},a_{t'}),\, g^+\rangle/\tau}}$$

g^+ = embedding of the *next* achievement unlocked after step t; t' = random step of the same episode; $\hat{g}$ = MLP on the FiLM-modulated latent; $\tau = 0.1$.

AD: cross-episode matching

Achievements of two episodes are paired by entropic partial optimal transport on $1 - \cos$ (reg 0.05; pairs with plan weight > 0.5). Same contrastive form: anchor g_A, positive = its match g_B, negative = a random achievement of B.

Both auxiliary losses add

$$+\,\beta_\pi\,\text{KL}\big(\pi_\theta \,\|\, \pi_{\text{before aux}}\big) + \beta_V\,(V_\theta(s) - \hat{R})^2, \quad \beta_\pi = \beta_V = 1$$

Our enhancement: a rarity bonus for rare achievements

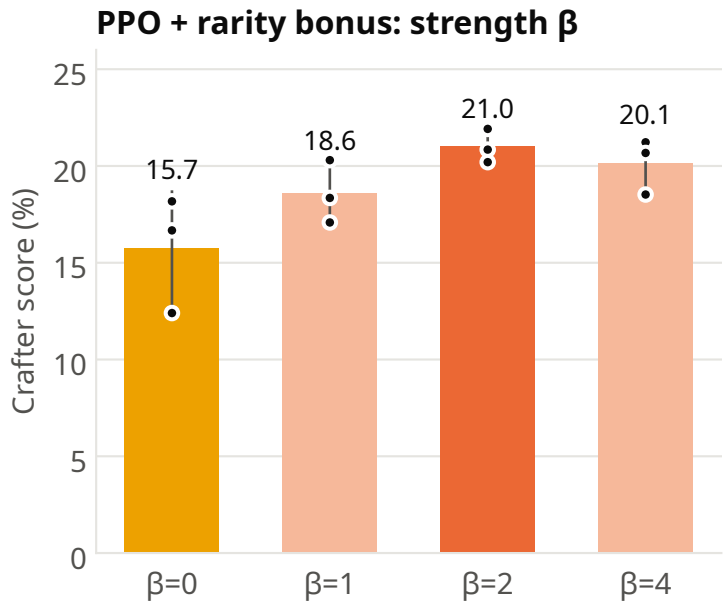
Pay more for what the agent rarely does. The bonus fades by itself as an achievement becomes common.

- When achievement i is unlocked for the first time in an episode, add **$\beta \cdot (1 - p_i)$**, where p_i is the share of the last 200 finished episodes that unlocked i.
- Common achievements stay worth ≈ +1; one never seen before is worth 1 + β. That fits the geometric-mean score.
- An automatic curriculum with no hand-written tech tree. About 25 lines (`envs.RarityBonus`), and it works with any algorithm.
- Logged rewards and the score always use the **game's** reward.

PPO, 3 seeds per β: β = 0 / 1 / 2 / 4 → 15.75 / 18.58 / **20.99** / 20.14.

β = 2 beats PPO on every seed (+8.4, +3.5, +3.7; paired t = 3.26, one-sided p ≈ 0.04), and the spread across seeds shrinks from ± 2.99 to ± 0.87.

On AD (seed 0): 17.71 → **23.08**. With 3 seeds: **23.65 ± 1.05**.



Results: ours beats PPO and the published AD result

1M steps. Training score = Crafter protocol (all training episodes). Evaluation = final weights on 200 fresh worlds per seed.

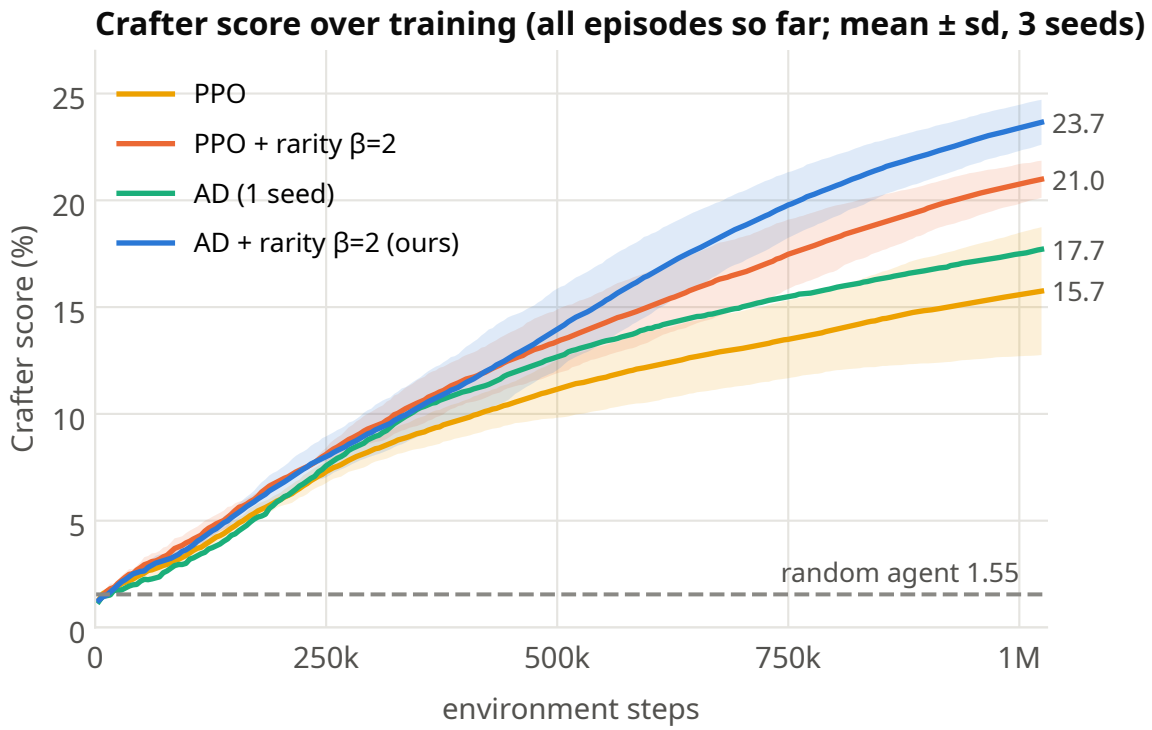
configuration	seeds	training score	training reward (all episodes)	eval score	eval reward
Random agent	–	1.55	1.41	1.5	1.4
PPO	3	15.75 ± 2.99	7.82 ± 0.33	21.1 ± 6.6	10.4 ± 1.0
PPO + rarity β=2	3	20.99 ± 0.87	8.61 ± 0.27	29.2 ± 0.9	11.6 ± 0.4
AD (paper config)	1	17.71	8.72	24.2	11.7
AD + rarity β=2 (ours)	**3**	**23.65 ± 1.05**	**9.28 ± 0.21**	**33.4 ± 0.5**	**13.0 ± 0.2**

Published references (Moon et al., 2023; 1M steps): PPO 15.60 ± 1.66, AD 21.79 ± 1.37 (10 seeds). Our PPO reproduces the paper (15.75); ours is ~1.9 points above published AD.

Caveat. Our own plain-AD run has 1 seed and scored low (17.71), so the AD vs AD + bonus gap is measured best on PPO (3 + 3 seeds). The compact submitted code was checked against the reference implementation (see Engineering).

Performance during training: Crafter score

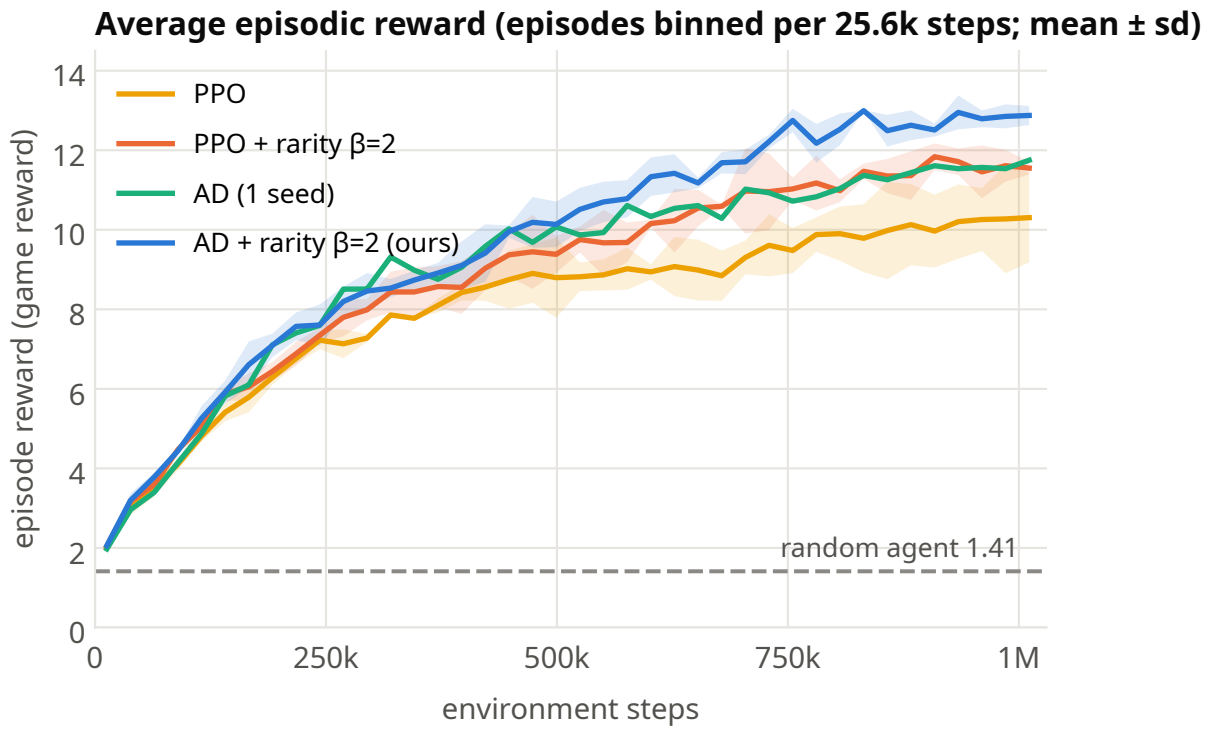
Cumulative score over all episodes so far, mean ± sd over seeds. The bonus pulls ahead after ~500k steps.



- All methods are identical for the first ~150k steps: easy achievements (wood, table, sapling) come first.
- The gap opens when the stone pickaxe and furnace are discovered, at 400–650k steps.
- The score at 1M is still rising: no method has converged within the budget.
- Ours ends at **23.7**; the last-100-episode score at the end is ≈ 33.

Average episodic reward

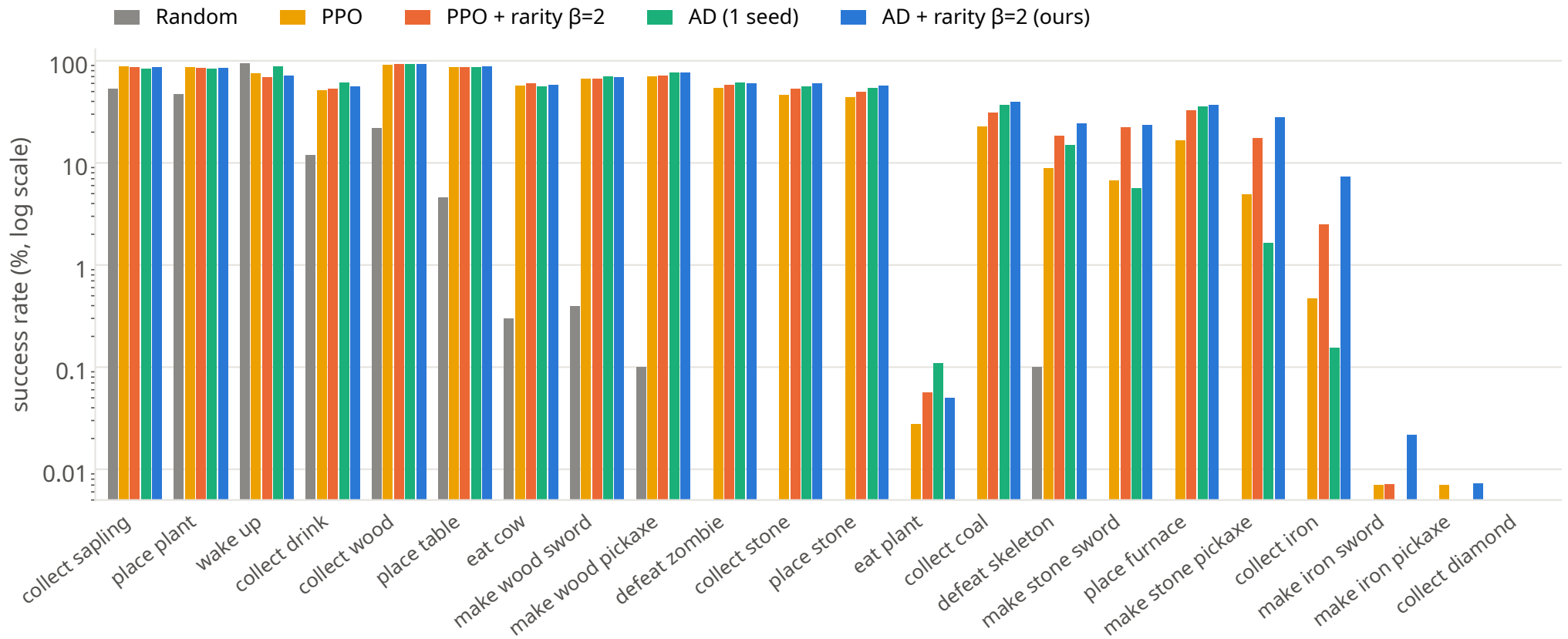
Game reward per episode (the bonus is not included), episodes binned per 25.6k steps, mean ± sd over seeds.



- Random agent: **1.41** per episode (1,008 episodes).
- Final reward (last 100 training episodes): PPO 10.3 ± 1.1, PPO + rarity 11.6 ± 0.2, ours **12.8 ± 0.2** (paper AD: 12.6).
- The reward rewards *how many* achievements an episode gets; the score rewards *which*. The bonus helps the score more than the reward, because it targets the rare ones.

Success rate on each skill (training, all 1M steps)

Log scale, as in the Crafter paper. Achievements ordered by depth in the tech tree.

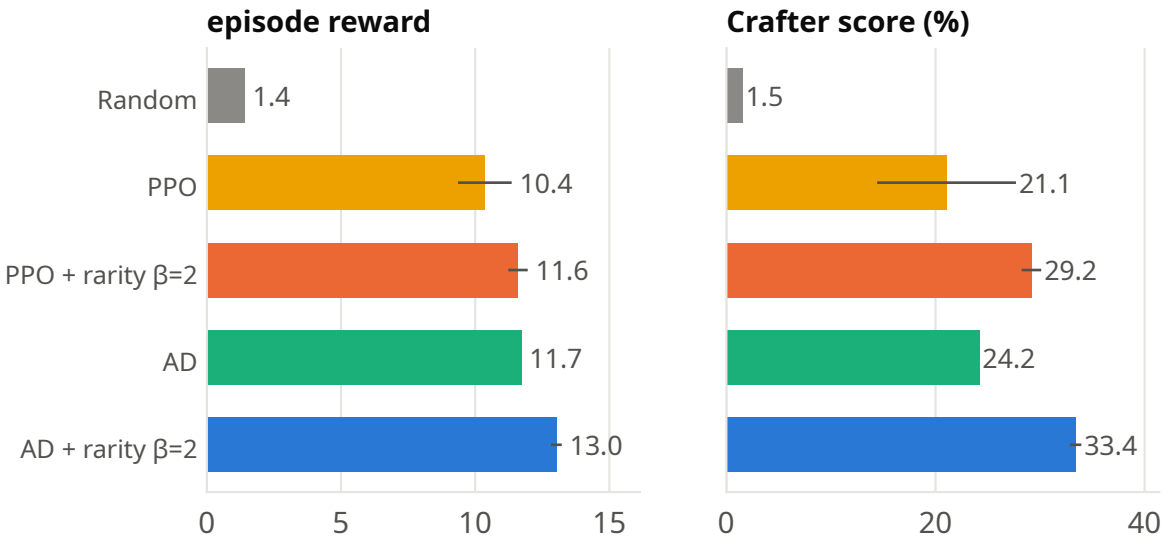


Ours vs PPO: stone pickaxe **28.0%** vs 4.9%, collect iron **7.4%** vs 0.5%, furnace 37.1% vs 16.7%.

Never or almost never solved by anyone: eat plant, iron tools, **diamond**. Cost of the bonus: wake up 71.7% vs 87.9% for plain AD.

Performance during evaluation: the final policies

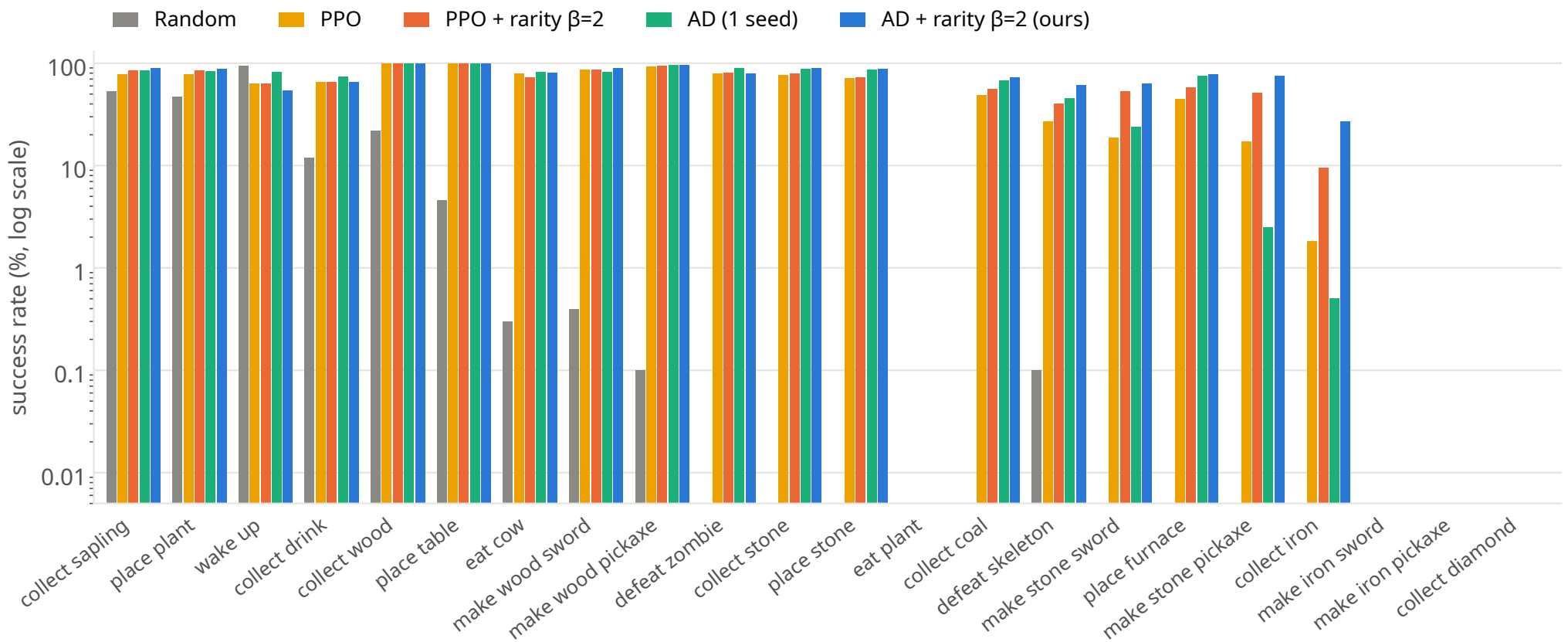
Final weights, stochastic policy as in training, 200 fresh worlds per seed (600 per configuration; plain AD: 200).



- The evaluation score is much higher than the training score, because it measures only the final policy and not the early learning.
- Ours: eval score **33.4 ± 0.5**, reward **13.0 ± 0.2**, mean length 251 steps.
- The ranking is the same as in training, and the seeds of ours agree closely (sd 0.5).
- Every evaluated episode ends in death (none reaches the 10,000-step limit). See "how the agents die".

Evaluation: skills of the final policies

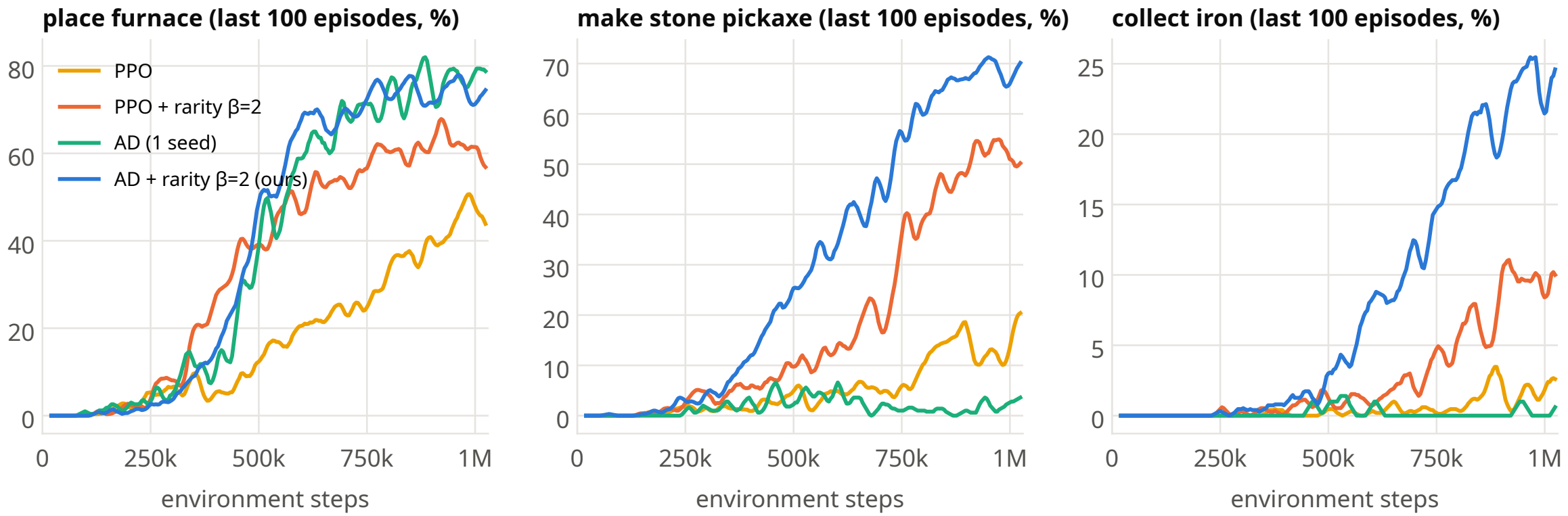
Success rate per achievement over fresh episodes with the final weights (log scale).



Our final agents make a stone pickaxe in most episodes and mine iron in about a third of them, but in 600 fresh episodes they never make an iron tool or find a diamond. In a larger search (800 episodes per agent) iron tools appear only a handful of times.

Why the bonus works: it breaks the stone-pickaxe wall

Success rate over the last 100 training episodes, mean over seeds.

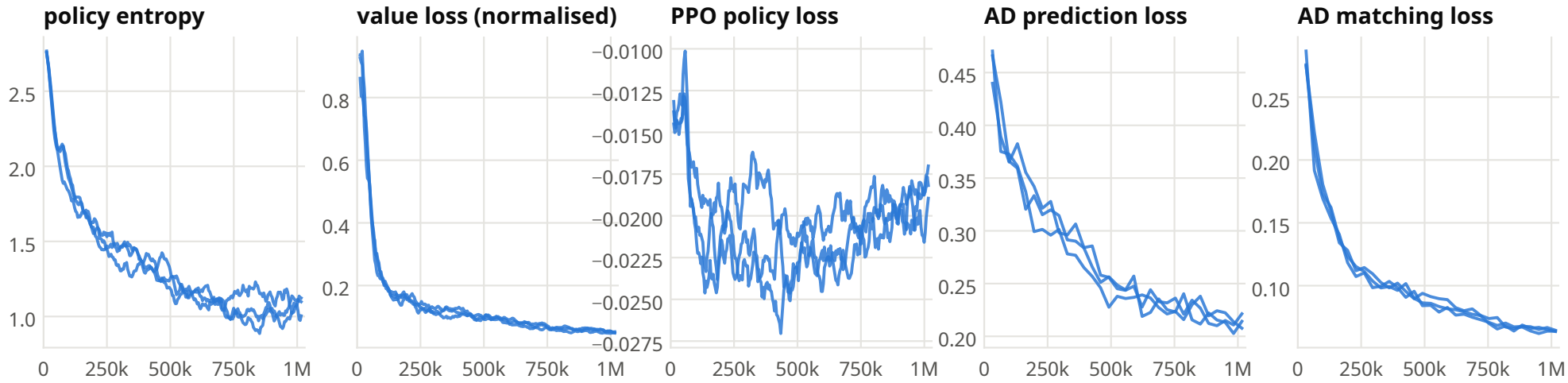


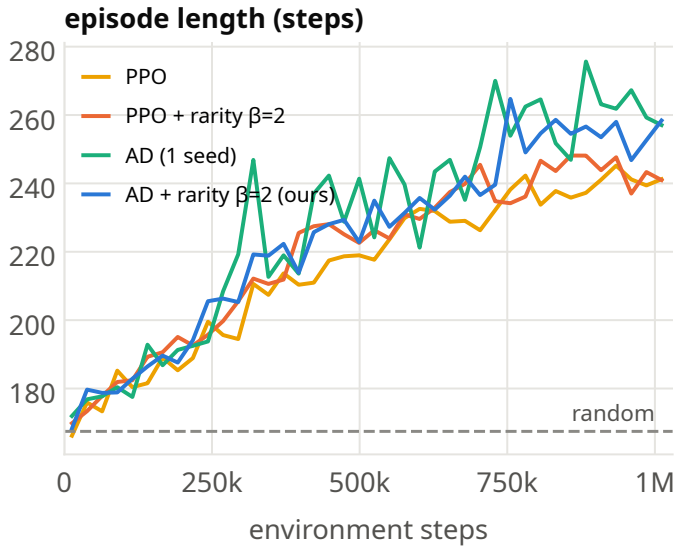
Without the bonus, agents learn the **stone sword** but stall at the **stone pickaxe** (1–2% for plain AD), although both need the same wood, stone and table. No pickaxe means no iron.

With the bonus, the first stone pickaxe is worth up to 3× a common achievement, so it is discovered earlier and reinforced: 70% of recent episodes for ours at the end, and iron follows.

Other metrics: losses, entropy and episode length

Ours, 3 seeds (one line each). The AD losses are logged once per auxiliary phase.

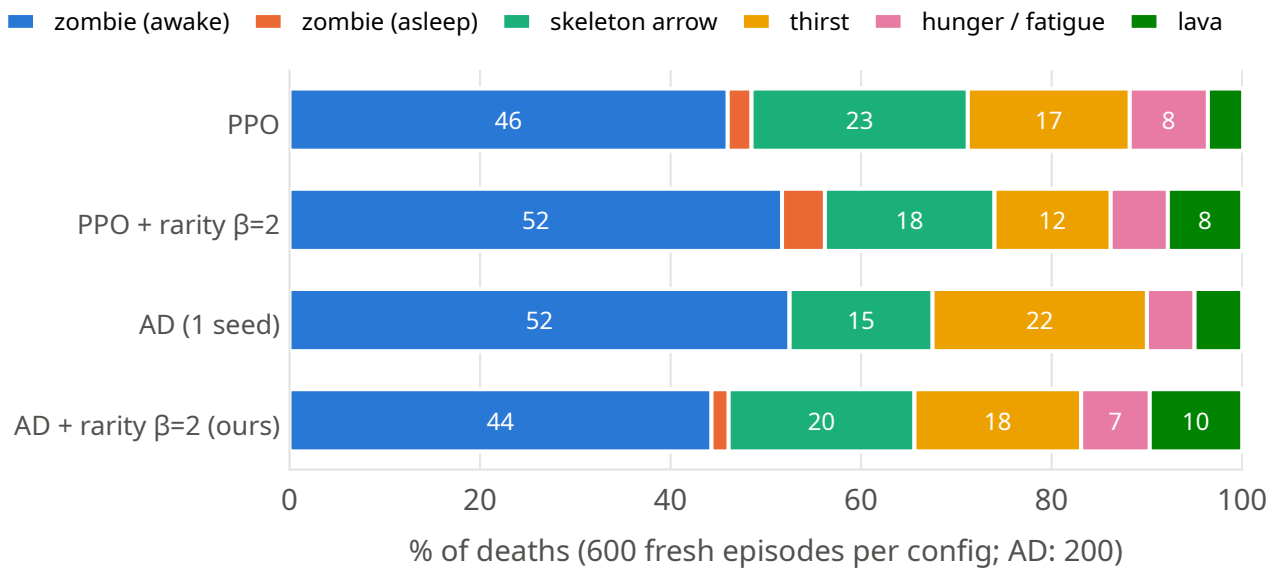




- Entropy falls from 2.8 (uniform over 17 actions = 2.83) to ≈ 1.0: the policy becomes more decisive but stays stochastic.
- Both contrastive losses keep falling: achievement embeddings become more predictable and more consistent across episodes.
- Episodes get longer as agents learn (~170 → ~250 steps), but most agents still die around the first night (a day lasts 300 steps).
- Wall-clock: ~3.6 h per AD seed on an RTX 5090 laptop GPU, 6.9 h on a 4 GB RTX 3050; PPO alone ~1.2 h.

Emergent behaviour: how the agents die

We hooked the game's damage code to log the cause of every lost health point (final weights, fresh worlds).



4,800 diagnosed episodes over 24 runs: 100% end in death, median length 220–260 steps.

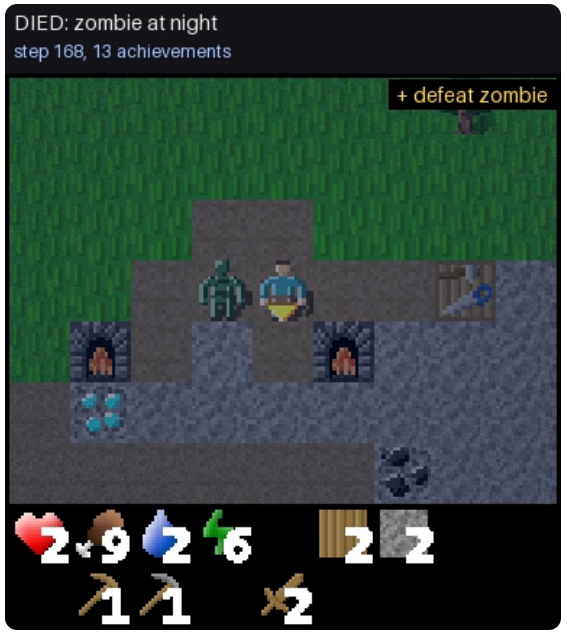
- **They stay awake at night.** Zombies cause about half of all deaths, mostly in the dark, but agents spend only ~4% of the night asleep.
- **They neglect their vitals.** At death, water is at 0 in 45% of episodes for ours; with food, water or energy at 0, health stops regenerating.
- **The bonus makes them sleep less:** episodes with any sleep fall from 84% (AD) to 58% (ours). Time goes to the tech tree instead.
- Arrows and lava (≈ 30%) hit them in caves, which is exactly where iron and diamonds are.

Emergent behaviour: what the agents do (and don't)

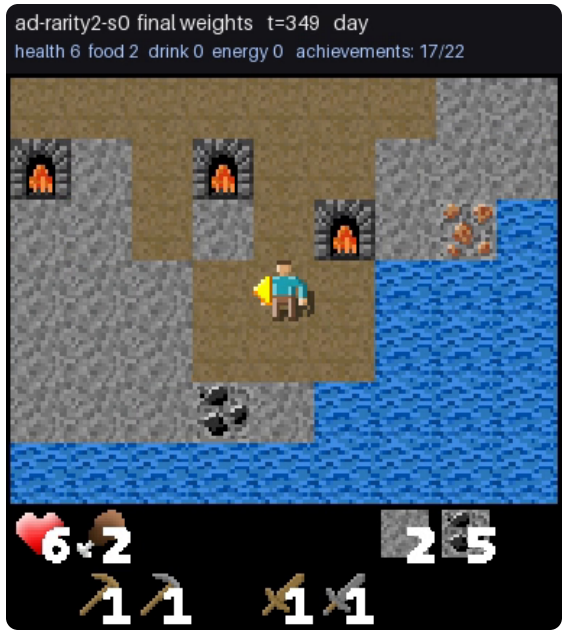
Frames from evaluation videos (final weights). The bar at the top shows the step, vitals and new achievements.



Iron pickaxe at night. Ours, seed 0: coal, furnace, iron and an iron pickaxe by step 178. A skeleton's arrow kills it at step 207.



Walks past diamonds. Killed by a zombie at night between two furnaces it built, a diamond one tile away (PPO + rarity, γ = 0.99).



Builds furnaces side by side. Ours, seed 0: three furnaces, water and energy at 0; it starves beside a lake at step 435.

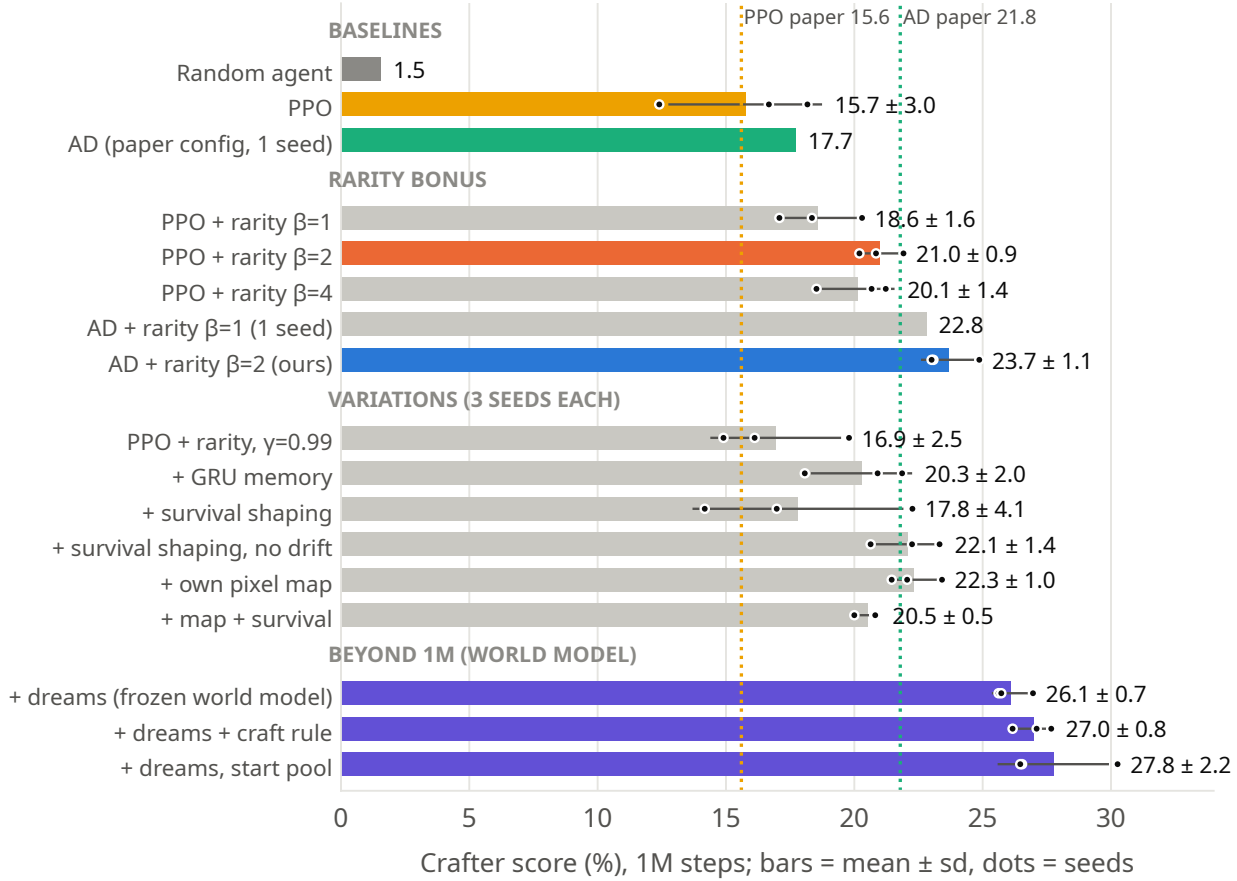


Iron tools, 1 health. With GRU memory: both iron tools at step 403, a diamond 4 tiles away, then dies of thirst and fatigue.

Also seen: a stone sword is learned long before a stone pickaxe (same ingredients); agents carry six stone swords; they press "make iron pickaxe" when the requirements are missing (6% of all actions).

What we tried on top, and what did not help

Each idea was motivated by the death analysis and tested on 3 seeds at 1M steps.



- **Longer horizon γ = 0.99** (nights last ~90 steps, γ = 0.95 sees ~20): 16.9 vs 21.0, worse on every seed, and survival is unchanged.
- **GRU memory** (remember where water and diamonds are): 20.3, learns more slowly.
- **Potential-based survival shaping** on food / water / energy: 17.8. It hides a $-(1-\gamma)\Phi$ cost per step alive; without it ("no drift") 22.1, the first change that makes agents *live longer* (+10% median life).
- **Own top-down map from pixels** (99% accurate odometry): 22.3. Agents see water in 95% of episodes but still don't go back: memory isn't the bottleneck.
- **Lesson:** within 1M steps, anything that adds learning signal or capacity not aimed at the tech tree slows it down.

Beyond the budget: practising in a learned world model

An exploration, **not a valid 1M result**: the world model was trained on 7k episodes (1.76M steps) recorded from our earlier agents.

- **World model** (ours, 25.6M parameters): each 7×7 game tile becomes a token (1,024 codes); a causal Transformer over 64 steps predicts the next frame (MaskGIT decoder), inventory, unlocks and death; a **top-down map memory** built from pixel odometry; a learned **craft rule** shared across tool tiers.
- **Dyna-style PPO:** each update also trains on 256 dreams × 16 steps, started from real moments near the iron chain. Dreams are sampled (T = 1) because greedy dreams are too safe.
- Score **26.1** → **27.8** (vs 23.7), iron collected in 15% of episodes (vs 7%), iron pickaxe up to 0.66% (vs 0.01%), and the first diamonds.

What we learned about dreams

• Before the craft rule, the model never dreamed an iron pickaxe: it had seen only 4 in its data (0 of 41 held-out crafts). With the shared rule: 97%.

• The real bottleneck for iron tools is *bringing a table and a furnace together* (2% of iron-in-hand moments), not knowing the recipe.

• Greedy dreams under-count damage ~4× at night; an agent trained on them would learn to ignore the night.

Engineering and reproducibility

What it took to run this on a 4 GB laptop GPU, and how the submitted code was checked.

- **Faster worlds:** Crafter's world generation took ~1 s per reset; with numba it takes 50 ms (identical maps). 8 envs: 180 → 1,620 steps/s.
- **4 GB VRAM:** gradient micro-batching (same gradient, value-normaliser EWMA adjusted to $\beta^{1/k}$); `--micro_batch 256`.
- **Faster AD auxiliary phase:** the frozen policy is evaluated once, not every epoch, and each distinct goal is encoded once: 934 s → 544 s, same math.
- **Results machine-checked:** the same configuration on two machines: 20.85 vs 20.90.

Submitted code (`train.py` , `model.py` , `algo.py` , `envs.py` , `evaluate.py` ; ~1,000 lines with comments, self-contained)

• `python train.py` trains the best configuration (AD + rarity β = 2, all paper hyperparameters).

• Checked against the reference AD implementation on a trained checkpoint: identical forward pass (difference 0.0); PPO and prediction losses and gradients equal to 1e-7, matching loss to 6e-5 (float rounding).

• Flags reproduce the ablations: `--no_ad` , `--rarity_bonus` , `--gamma` , `--vitals` .

Conclusions

- **A rarity bonus is a cheap, effective fix for Crafter's bottleneck.** PPO 15.75 → 20.99 (3/3 seeds better), and AD + bonus reaches **23.65 ± 1.05**, above the published AD (21.79).
- It works by breaking the **stone-pickaxe wall**, which opens coal → furnace → iron.
- The best agents now die of **neglected vitals and zombies at night**, not missing skills. Horizon, memory and survival shaping each cost tech-tree progress within 1M steps.
- Diamonds stay out of reach at 1M steps. Learning in a world model's dreams lifts iron tools ×6–60, but only with data beyond the budget.

Next steps

• A world model learned online from the agent's own 1M steps (honest Dyna).

• Rewarding "table and furnace together" as a sub-goal.

• A bonus that is aware of the tech tree (pickaxe before sword).

Appendix: hyperparameters (defaults of `train.py`)

All AD hyperparameters are the paper's; the rarity bonus is ours.

parameter	value
environments × steps per rollout	8 × 512 = 4,096
updates (1M steps)	245
discount γ / GAE λ	0.95 / 0.65
PPO epochs × minibatches	3 × 8 (512 samples)
clip ε / value coef / entropy coef	0.2 / 0.5 / 0.01
Adam learning rate, grad-norm clip	3e-4, 0.5
network	IMPALA [64,128,128] × 2 blocks
latent / heads input	1024 / 2048, 9.23M params

parameter	value
AD phase every / epochs	8 updates / 6
contrastive temperature τ	0.1
KL and value distillation coefs	1.0 / 1.0
OT regularisation, targets per source	0.05, 16 episodes
rarity bonus β / window	**2.0 / 200 episodes**
vitals shaping α (ablation)	0 (off)
seeds	0, 1, 2
original runs	250 updates (1.024M steps)

Appendix: every configuration, per seed

Crafter score over all training episodes (1M steps).

configuration	seeds	per seed	mean ± sd
Random agent	1	1.55	1.55
PPO	3	12.40 / 16.67 / 18.17	15.75 ± 2.99
AD (paper config, 1 seed)	1	17.71	17.71
PPO + rarity β=1	3	20.30 / 18.35 / 17.08	18.58 ± 1.62
PPO + rarity β=2	3	20.85 / 20.20 / 21.92	20.99 ± 0.87
PPO + rarity β=4	3	21.22 / 18.53 / 20.67	20.14 ± 1.43
AD + rarity β=1 (1 seed)	1	22.80	22.80
AD + rarity β=2 (ours)	3	23.08 / 24.87 / 23.01	23.65 ± 1.05
PPO + rarity, γ=0.99	3	19.79 / 14.90 / 16.12	16.94 ± 2.55
+ GRU memory	3	20.91 / 21.86 / 18.07	20.28 ± 1.97
+ survival shaping	3	22.26 / 16.98 / 14.17	17.80 ± 4.11
+ survival shaping, no drift	3	20.64 / 23.32 / 22.25	22.07 ± 1.35
+ own pixel map	3	21.46 / 22.06 / 23.42	22.31 ± 1.00
+ map + survival	3	20.00 / 20.78 / 20.81	20.53 ± 0.46
+ dreams (frozen world model)	3	25.62 / 25.73 / 26.96	26.10 ± 0.75
+ dreams + craft rule	3	27.67 / 26.17 / 27.10	26.98 ± 0.76
+ dreams, start pool	3	26.53 / 30.26 / 26.47	27.75 ± 2.17